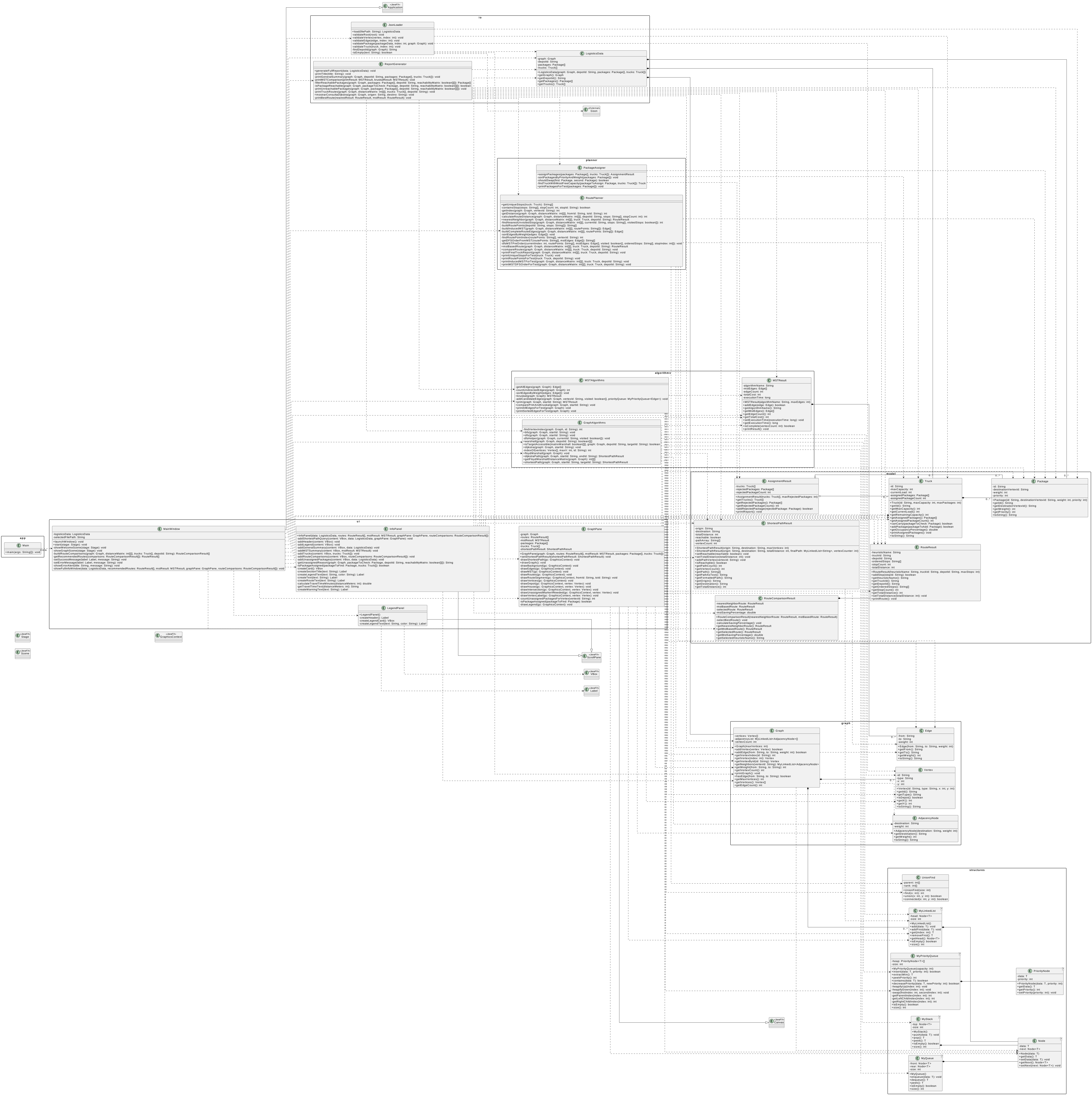
Diagrama UML completo - LogisTEC


«JavaFX» Application
io
JsonLoader
+load(filePath: String): LogisticsData
-validateRoot(root): void
-validateVertex(vertex, index: int): void
-validateEdge(edge, index: int): void
-validatePackage(packageData, index: int, graph: Graph): void
-validateTruck(truck, index: int): void
-findDepotId(graph: Graph): String
-isEmpty(text: String): boolean
ReportGenerator
+generateFullReport(data: LogisticsData): void
-printTitle(title: String): void
-printGeneralSummary(graph: Graph, depotId: String, packages: Package[], trucks: Truck[]): void
-printMSTComparison(primResult: MSTResult, kruskalResult: MSTResult): void
-filterReachablePackages(graph: Graph, packages: Package[], depotId: String, reachabilityMatrix: boolean[][]): Package[]
-isPackageReachable(graph: Graph, packageToCheck: Package, depotId: String, reachabilityMatrix: boolean[][]): boolean
-printUnreachablePackages(graph: Graph, packages: Package[], depotId: String, reachabilityMatrix: boolean[][]): void
-printTruckRoutes(graph: Graph, distanceMatrix: int[][], trucks: Truck[], depotId: String): void
+mostrarConsultaDijkstra(graph: Graph, origen: String, destino: String): void
-printBestRoute(nearestResult: RouteResult, mstResult: RouteResult): void
LogisticsData
-graph: Graph
-depotId: String
-packages: Package[]
-trucks: Truck[]
+LogisticsData(graph: Graph, depotId: String, packages: Package[], trucks: Truck[])
+getGraph(): Graph
+getDepotId(): String
+getPackages(): Package[]
+getTrucks(): Truck[]
«External» Gson
planner
PackageAssigner
+assignPackages(packages: Package[], trucks: Truck[]): AssignmentResult
-sortPackagesByPriorityAndWeight(packages: Package[]): void
-shouldSwap(first: Package, second: Package): boolean
-findTruckWithMostFreeCapacity(packageToAssign: Package, trucks: Truck[]): Truck
+printPackagesForTest(packages: Package[]): void
RoutePlanner
+getUniqueStops(truck: Truck): String[]
-containsStop(stops: String[], stopCount: int, stopId: String): boolean
-getIndex(graph: Graph, vertexId: String): int
-getDistance(graph: Graph, distanceMatrix: int[][], fromId: String, toId: String): int
-calculateRouteDistance(graph: Graph, distanceMatrix: int[][], depotId: String, stops: String[], stopCount: int): int
+nearestNeighbor(graph: Graph, distanceMatrix: int[][], truck: Truck, depotId: String): RouteResult
-findNearestUnvisitedStop(graph: Graph, distanceMatrix: int[][], currentId: String, stops: String[], visitedStops: boolean[]): int
-buildRoutePoints(depotId: String, stops: String[]): String[]
-buildInducedMST(graph: Graph, distanceMatrix: int[][], routePoints: String[]): Edge[]
-buildCompleteRouteEdges(graph: Graph, distanceMatrix: int[][], routePoints: String[]): Edge[]
-sortEdgesByWeight(edges: Edge[]): void
-findRoutePointIndex(routePoints: String[], vertexId: String): int
-getDFSOrderFromMST(routePoints: String[], mstEdges: Edge[]): String[]
-dfsMSTPreOrder(currentIndex: int, routePoints: String[], mstEdges: Edge[], visited: boolean[], orderedStops: String[], stopIndex: int[]): void
+mstBasedRoute(graph: Graph, distanceMatrix: int[][], truck: Truck, depotId: String): RouteResult
+compareRoutes(graph: Graph, distanceMatrix: int[][], truck: Truck, depotId: String): void
+printFinalTruckReport(graph: Graph, distanceMatrix: int[][], truck: Truck, depotId: String): void
+printUniqueStopsForTest(truck: Truck): void
+printRoutePointsForTest(truck: Truck, depotId: String): void
+printInducedMSTForTest(graph: Graph, distanceMatrix: int[][], truck: Truck, depotId: String): void
+printMSTDFSOrderForTest(graph: Graph, distanceMatrix: int[][], truck: Truck, depotId: String): void
algorithms
MSTAlgorithms
-getAllEdges(graph: Graph): Edge[]
-countUndirectedEdges(graph: Graph): int
-sortEdgesByWeight(edges: Edge[]): void
+kruskal(graph: Graph): MSTResult
-addCandidateEdges(graph: Graph, vertexId: String, visited: boolean[], priorityQueue: MyPriorityQueue<Edge>): void
+prim(graph: Graph, startId: String): MSTResult
+comparePrimAndKruskal(graph: Graph, startId: String): void
+printAllEdgesForTest(graph: Graph): void
+printSortedEdgesForTest(graph: Graph): void
GraphAlgorithms
-findVertexIndex(graph: Graph, id: String): int
+bfs(graph: Graph, startId: String): void
+dfs(graph: Graph, startId: String): void
-dfsHelper(graph: Graph, currentId: String, visited: boolean[]): void
+warshall(graph: Graph, depotId: String): boolean[][]
+isTargetAccessible(matrixWarshall: boolean[][], graph: Graph, depotId: String, targetId: String): boolean
+dijkstra(graph: Graph, startId: String): void
-indexOf(vertices: Vertex[], maxV: int, id: String): int
+floydWarshall(graph: Graph): void
+dijkstraPath(graph: Graph, startId: String, endId: String): ShortestPathResult
+getFloydWarshallDistanceMatrix(graph: Graph): int[][]
+shortestPath(graph: Graph, startId: String, targetId: String): ShortestPathResult
MSTResult
-algorithmName: String
-mstEdges: Edge[]
-edgeCount: int
-totalCost: int
-executionTime: long
+MSTResult(algorithmName: String, maxEdges: int)
+addEdge(edge: Edge): boolean
+getAlgorithmName(): String
+getMstEdges(): Edge[]
+getEdgeCount(): int
+getTotalCost(): int
+setExecutionTime(executionTime: long): void
+getExecutionTime(): long
+isComplete(vertexCount: int): boolean
+printResult(): void
app
Main
+main(args: String[]): void
inicia
ui
MainWindow
-logisticsData: LogisticsData
-selectedFilePath: String
+launchWindow(): void
+start(stage: Stage): void
-showWelcomeScene(stage: Stage): void
-showGraphScene(stage: Stage): void
-buildRouteComparisons(graph: Graph, distanceMatrix: int[][], trucks: Truck[], depotId: String): RouteComparisonResult[]
-getRecommendedRoutes(comparisons: RouteComparisonResult[]): RouteResult[]
-setSuccessMessage(label: Label, message: String): void
-setErrorMessage(label: Label, message: String): void
-showErrorAlert(title: String, message: String): void
-showFullInfoWindow(data: LogisticsData, recommendedRoutes: RouteResult[], mstResult: MSTResult, graphPane: GraphPane, routeComparisons: RouteComparisonResult[]): void
InfoPanel
+InfoPanel(data: LogisticsData, routes: RouteResult[], mstResult: MSTResult, graphPane: GraphPane, routeComparisons: RouteComparisonResult[])
-addShortestPathQuery(content: VBox, data: LogisticsData, graphPane: GraphPane): void
-addHeader(content: VBox): void
-addLegend(content: VBox): void
-addGeneralSummary(content: VBox, data: LogisticsData): void
-addMSTSummary(content: VBox, mstResult: MSTResult): void
-addTrucks(content: VBox, trucks: Truck[]): void
-addRouteComparisons(content: VBox, routeComparisons: RouteComparisonResult[]): void
-addUnassignedPackages(content: VBox, data: LogisticsData): void
-getUnassignedReason(graph: Graph, packageToCheck: Package, depotId: String, reachabilityMatrix: boolean[][]): String
-isPackageAssigned(packageToFind: Package, trucks: Truck[]): boolean
-createCard(): VBox
-createSectionTitle(text: String): Label
-createLegendText(text: String, color: String): Label
-createText(text: String): Label
-createRouteText(text: String): Label
-calculateTravelTimeMinutes(distanceMeters: int): double
-getTravelTimeText(distanceMeters: int): String
-createWarningText(text: String): Label
GraphPane
-graph: Graph
-routes: RouteResult[]
-mstResult: MSTResult
-packages: Package[]
-trucks: Truck[]
-shortestPathResult: ShortestPathResult
+GraphPane(graph: Graph, routes: RouteResult[], mstResult: MSTResult, packages: Package[], trucks: Truck[])
+setShortestPathResult(shortestPathResult: ShortestPathResult): void
-drawShortestPath(gc: GraphicsContext): void
+drawGraph(): void
-drawBackground(gc: GraphicsContext): void
-drawEdges(gc: GraphicsContext): void
-drawMST(gc: GraphicsContext): void
-drawRoutes(gc: GraphicsContext): void
-drawRouteSegment(gc: GraphicsContext, fromId: String, toId: String): void
-drawVertices(gc: GraphicsContext): void
-drawDepot(gc: GraphicsContext, vertex: Vertex): void
-drawHouse(gc: GraphicsContext, vertex: Vertex): void
-drawIntersection(gc: GraphicsContext, vertex: Vertex): void
-drawUnassignedMarkerIfNeeded(gc: GraphicsContext, vertex: Vertex): void
-drawVertexLabel(gc: GraphicsContext, vertex: Vertex): void
-countUnassignedPackagesForVertex(vertexId: String): int
-isPackageAssigned(packageToFind: Package): boolean
-drawLegend(gc: GraphicsContext): void
LegendPanel
+LegendPanel()
-createHeader(): Label
-createLegendCard(): VBox
-createLegendText(text: String, color: String): Label
«JavaFX» Stage
«JavaFX» Scene
«JavaFX» GraphicsContext
«JavaFX» ScrollPane
«JavaFX» VBox
«JavaFX» Label
«JavaFX» Canvas
model
AssignmentResult
-trucks: Truck[]
-rejectedPackages: Package[]
-rejectedPackageCount: int
+AssignmentResult(trucks: Truck[], maxRejectedPackages: int)
+getTrucks(): Truck[]
+getRejectedPackages(): Package[]
+getRejectedPackageCount(): int
+addRejectedPackage(rejectedPackage: Package): boolean
+printReport(): void
Truck
-id: String
-maxCapacity: int
-currentLoad: int
-assignedPackages: Package[]
-assignedPackageCount: int
+Truck(id: String, maxCapacity: int, maxPackages: int)
+getId(): String
+getMaxCapacity(): int
+getCurrentLoad(): int
+getRemainingCapacity(): int
+getAssignedPackages(): Package[]
+getAssignedPackageCount(): int
+canCarry(packageToCheck: Package): boolean
+addPackage(packageToAdd: Package): boolean
+getOccupancyPercentage(): double
+printAssignedPackages(): void
+toString(): String
Package
-id: String
-destinationVertexId: String
-weight: int
-priority: int
+Package(id: String, destinationVertexId: String, weight: int, priority: int)
+getId(): String
+getDestinationVertexId(): String
+getWeight(): int
+getPriority(): int
+toString(): String
ShortestPathResult
-origin: String
-destination: String
-totalDistance: int
-reachable: boolean
-pathArray: String[]
-vertexCount: int
+ShortestPathResult(origin: String, destination: String, maxVertices: int)
+ShortestPathResult(origin: String, destination: String, totalDistance: int, finalPath: MyLinkedList<String>, vertexCounter: int)
+setReachable(reachable: boolean): void
+setTotalDistance(totalDistance: int): void
+addPathVertex(vertexId: String): void
+isReachable(): boolean
+getPathCount(): int
+getVertexCount(): int
+getPath(): String[]
+getPathAsText(): String
+getFormattedPath(): String
+getOrigin(): String
+getDestination(): String
+getTotalDistance(): int
RouteResult
-heuristicName: String
-truckId: String
-depotId: String
-orderedStops: String[]
-stopCount: int
-totalDistance: int
+RouteResult(heuristicName: String, truckId: String, depotId: String, maxStops: int)
+addStop(stopId: String): boolean
+getHeuristicName(): String
+getTruckId(): String
+getDepotId(): String
+getOrderedStops(): String[]
+getStopCount(): int
+getTotalDistance(): int
+setTotalDistance(totalDistance: int): void
+printRoute(): void
RouteComparisonResult
-nearestNeighborRoute: RouteResult
-mstBasedRoute: RouteResult
-selectedRoute: RouteResult
-mstSavingPercentage: double
+RouteComparisonResult(nearestNeighborRoute: RouteResult, mstBasedRoute: RouteResult)
-selectBestRoute(): void
-calculateSavingPercentage(): void
+getNearestNeighborRoute(): RouteResult
+getMstBasedRoute(): RouteResult
+getSelectedRoute(): RouteResult
+getMstSavingPercentage(): double
+getSelectedHeuristicName(): String
0..*
graph
Graph
-vertices: Vertex[]
-adjacencyList: MyLinkedList<AdjacencyNode>[]
-vertexCount: int
+Graph(maxVertices: int)
+addVertex(vertex: Vertex): boolean
+addEdge(from: String, to: String, weight: int): boolean
+getVertexIndex(id: String): int
+getVertex(index: int): Vertex
+getVertexById(id: String): Vertex
+getNeighbors(vertexId: String): MyLinkedList<AdjacencyNode>
+getWeight(from: String, to: String): int
+getVertexCount(): int
+printGraph(): void
+hasEdge(from: String, to: String): boolean
+getMaxVertices(): int
+getVertices(): Vertex[]
+getEdgeCount(): int
Edge
-from: String
-to: String
-weight: int
+Edge(from: String, to: String, weight: int)
+getFrom(): String
+getTo(): String
+getWeight(): int
+toString(): String
Vertex
-id: String
-type: String
-x: int
-y: int
+Vertex(id: String, type: String, x: int, y: int)
+getId(): String
+getType(): String
+isDepot(): boolean
+getX(): int
+getY(): int
+toString(): String
AdjacencyNode
-destination: String
-weight: int
+AdjacencyNode(destination: String, weight: int)
+getDestination(): String
+getWeight(): int
+toString(): String
structures
UnionFind
-parent: int[]
-rank: int[]
+UnionFind(size: int)
+find(x: int): int
+union(x: int, y: int): boolean
+connected(x: int, y: int): boolean
MyLinkedList T
-head: Node<T>
-size: int
+MyLinkedList()
+add(data: T): void
+addFirst(data: T): void
+get(index: int): T
+removeFirst(): T
+getHead(): Node<T>
+isEmpty(): boolean
+size(): int
MyPriorityQueue T
-heap: PriorityNode<T>[]
-size: int
+MyPriorityQueue(capacity: int)
+insert(data: T, priority: int): boolean
+extractMin(): T
+peekPriority(): int
+contains(data: T): boolean
+decreasePriority(data: T, newPriority: int): boolean
-heapifyUp(index: int): void
-heapifyDown(index: int): void
-swap(firstIndex: int, secondIndex: int): void
-getParentIndex(index: int): int
-getLeftChildIndex(index: int): int
-getRightChildIndex(index: int): int
+isEmpty(): boolean
+size(): int
PriorityNode T
-data: T
-priority: int
+PriorityNode(data: T, priority: int)
+getData(): T
+getPriority(): int
+setPriority(priority: int): void
MyStack T
-top: Node<T>
-size: int
+MyStack()
+push(data: T): void
+pop(): T
+peek(): T
+isEmpty(): boolean
+size(): int
Node T
-data: T
-next: Node<T>
+Node(data: T)
+getData(): T
+setData(data: T): void
+getNext(): Node<T>
+setNext(next: Node<T>): void
MyQueue T
-front: Node<T>
-rear: Node<T>
-size: int
+MyQueue()
+enqueue(data: T): void
+dequeue(): T
+peek(): T
+isEmpty(): boolean
+size(): int